NLH Platform — Program Audit (read-only)

Repository	`nlhnagpur/nlh-platform` (working copy `D:\Claude\nlh-platform`)
Code state audited	`main` @ `332a4f3` (2026-10-12)
Database audited	Supabase project `frnnoxudtlvhyyoqdqzx` — schema, constraints, triggers and RLS helper functions read live; row counts as of 2026-10-12
Purpose	Factual map of inputs, outputs, data model, rules and UI, to be used by a later redesign session. No redesign is proposed here.
Method	Source read for every claim; each claim cites `file › function` (and line, where the line is stable).

Evidence labels used throughout

- **[C]** — confirmed by reading code or the live database.
- **[S]** — taken from the five reference screenshot descriptions in the brief. *The image files themselves were not attached to this session; only their text descriptions were available, so every [S] item was cross-checked against code where possible and discrepancies are listed in §8.*
- **[I]** — inferred; reasoning stated.

Scale of the system (live row counts, 2026-10-12): 37 students · 81 enrolments · 28 franchisees · 49 logins · 59 courses / 65 SKUs · 30 batches · 917 sessions · 2,172 attendance rows · 18 student invoices · 42 student payments · 18 instructors · 2 coupons · 31 kit orders · 121 WhatsApp messages. [C]

---

1. System overview

1.1 Modules

Module (sidebar id)	Page file	What it is for
Dashboard (`dashboard`)	`src/pages/DashboardPage.jsx`	Role-specific KPIs, recent orders, follow-ups, quick actions
Franchisees (`franchisees`)	`FranchiseesPage.jsx`	Franchisee tree (SMF → CF → UF, plus SCHOOL), onboarding, fees, courses, agreements, certificates
Orders (`orders`)	`OrdersPage.jsx`	Kit orders from franchisees → proforma → invoice → payment → dispatch; returns/credit notes
Students (`students`)	`StudentsPage.jsx` (+ `AttendanceRegister.jsx`, `AttendanceSheet.jsx`, `StudentCertModal.jsx`, `StudentLedgerView.jsx`)	Admission, enrolment, fees, invoices, receipts, attendance, completion, certificates
Instructors (`instructors`)	`InstructorsPage.jsx`	Course Instructors (CIs): appointments, rates, caution deposit, payroll, batches
Batches (`batches`)	`BatchesPage.jsx`	Batch set-up, roster, session attendance, history
WhatsApp Inbox (`whatsapp-inbox`)	`WhatsAppInboxPage.jsx`	Inbound/outbound log of the business number; free-text replies
Franchisee chat (`messages`)	`MessagesPage.jsx`	In-app messaging HO ↔ franchisee, broadcasts, attachments
HO Accounting (`accounting`)	`AccountingPage.jsx`	Chart of accounts, expenses, journal, commission splits, vendors, banks
Courses & SKUs (`courses`)	`CoursesPage.jsx`	Programs, levels, fees, rates, session totals

Module (sidebar id)	Page file	What it is for
Discount coupons (`coupons`)	`CouponsPage.jsx`	Coupon definitions and limits
Inventory & Kits (`inventory`)	`InventoryPage.jsx` (+ `KitEditorModal.jsx`)	Items, stock ledger, kit definitions per SKU
Kit prices / Price log (`prices`, `price-history`)	`PricesPage.jsx`, `PriceHistoryPage.jsx`	Kit pricing and its audit log
Email log (`email-log`)	`EmailLogPage.jsx`	Log of transactional emails
Manage logins (`users`)	`UsersPage.jsx`	Logins, roles, staff permission sets, block/delete, invites
Access requests (`requests`)	`AccessRequestsPage.jsx`	Public "request platform access" form and approval
My Account (`ledger`)	`MyAccountPage.jsx` (+ `FranchiseeLedgerView.jsx`)	A franchisee's own statement of account
Landing / Login / Reset / Onboarding	`LandingPage.jsx`, `LoginPage.jsx`, `ResetPasswordPage.jsx`, `OnboardingPage.jsx`	Pre-login and first-login flows

Routing is not URL-based: `AppShell.jsx` holds `currentPage` state and maps the sidebar id to a lazy-loaded page via `PAGE_MAP` (L26–45); a page is blocked unless `can(currentPage + '.view')` (L119). Only `/login` and `/apply` are real URLs (`AuthContext.boot`). [C]

1.2 Roles

Source: `src/constants/roles.js` (`ADMIN_ROLES`, `NAV_ITEMS`), `src/constants/permissions.js`, `src/context/AuthContext.jsx`, DB check `users_role_check`. [C]

Group	Roles	Scope
Head Office (admin)	`owner`, `super_admin` (same nav), `admin`, `manager`	Everything. `manager` has no chat / logins / requests / price-log pages
Head Office (limited)	`staff`	Pages and actions chosen per login (`users.permissions` JSON; presets in `PRESETS`)
Franchise tree	`smf`, `cf`, `uf`	Own centre plus descendants (RLS `nlh_accessible_franchisee_ids()`); fewer nav items
Other	`student`, `school`	`student` → onboarding / "My courses"; `school` exists in the DB role check but has no nav entry; schools log in as `uf` (`FranchiseesPage.resendAccessImpl` role map)

`isAdminRole()` is the front-end gate; `nlh_is_admin()` and `nlh_staff_can(keys[])` are the database-side equivalents (live function definitions read). A staff login with no ticked key is denied by RLS; non-staff always passes `nlh_staff_can`. [C]

1.3 Franchise hierarchy and the student's place in it

```
NLH Head Office (tier NLH, centre_code HO)
 └ SMF  State Master Franchisee   – 1 per Indian state / foreign country
    └ CF  City Franchisee         – 1 per city
       └ UF  Unit Franchisee      – many per city
          └ SCHOOL                – tier 'SCHOOL', parented under a CF (B2B customer)
             └ Students (students.franchisee_id → franchisees.id)
```

Territory uniqueness is enforced in the **client** only (`FranchiseesPage.AddFranchiseeModal.save`, `OnboardingPage.submitFranchisee`); there is no database constraint for it. [C]

1.4 Runtime architecture

- **Front end:** React 18 + Vite SPA; state is local `useState`; no router, no global store beyond `AuthContext`. [C]
- **Data access:** the browser talks to Supabase directly (anon key + RLS) for almost everything; money rules are enforced partly in the browser, partly in DB triggers (see §5). [C]
- **Server functions (`api/*.js`, Vercel):** `create-user`, `change-user-email`, `send-email`, `send-whatsapp`, `send-payment-whatsapp`, `send-review-whatsapp`, `send-cert-whatsapp`, `whatsapp-webhook`, `create-razorpay-order`, `razorpay-webhook`, shared `_auth.js`. [C]
- **External services:** Brevo (email), Meta WhatsApp Cloud API (business number +91 712 351 4575), Razorpay (online payment of orders), BoldSign (agreement signature; `supabase/functions/boldsign-webhook`), headless Chromium on Vercel to render the certificate image (`api/send-cert-whatsapp.js`). [C]
- **Dual-write ledger:** `transactions`, `transaction_items`, `transaction_payments` are written alongside the primary money tables (`StudentsPage.mirrorStudentToTransaction` L37, `mirrorStudentPayment` L89; `OrdersPage` L62–84; `FranchiseesPage` L198–211). Nothing in `src` reads them back (grep over `from('transactions')`), so they are a write-only mirror today. Migration plan: `docs/transaction-model-migration-plan.md`. [C]

---

2. Inputs

Legend: **R** required (blocked by code), **O** optional. Validation is what the code actually checks; "—" means none.

2.1 Access request (public form) — `AccessRequestsPage › PublicRequestForm.submit`

Field	Type	R/O	Validation
first_name, last_name	text	R	non-empty
business_name	text	O	—
email	email	R	contains `@`
phone	text	R	non-empty (no digit-count check)
role_requested	select `uf/cf/smf/staff`	R	default `uf`
programs_requested	multi-select of active `courses.group_name`	R only when role = `uf`	≥ 1
address, area, city, pincode, state	text	R	non-empty
date_of_birth	date	R	non-empty
qualification	text	O	—

2.2 Self-onboarding after first login — `OnboardingPage`

Path	Fields	Required	Notes
Franchisee (`submitFranchisee`)	tier, name, email, phone, country, state, city, area, address, parent	name, email, phone; parent for UF/CF	SMF/CF territory duplicate check (client). Inserts `franchisees`; trigger `tr_franchisee_onboarded` sets the login's role/franchisee
Student (`submitStudent`)	student name, parent name, phone, email, DOB, country, state, city, area, address, centre	name, phone, centre	Sets `fee_total 0`, `payment_status 'none'`; links `users.student_id`

2.3 Franchisee admin — `FranchiseesPage`

AddFranchiseeModal.save (L1810): owner_name R, email R, tier R, parent (UF/CF) R; business_name, phone, country, state, city, area, pincode, address O. Side effects: courses defaulted (SMF/CF = all active, UF = none), contract 3 years from today, `/api/create-user` with generated temp password, welcome email + welcome letter. [C]

FranchiseeDetailModal (L367) Profile/fee fields (`form` state): name, owner_name, email (changed through `changeEmail` → `/api/change-user-email`), phone, phone2, country/state/city/area/pincode/address, status, enrollment_fee, fee_paid, enrollment_date (→ `contract_start`), valid_till, renewal_fee, date_of_birth, qualification, registered_courses (checkbox list). `save()` L728 writes them all in one update. Tabs: info, courses, orders, students, ledger, cert, agreement, cis, commission, schools. [C]

RecordFranchiseePaymentModal: amount (default = balance), date, mode, reference; DB trigger `check_franchisee_payment_total` rejects a total above `enrollment_fee`. [C]

2.4 Courses & SKUs — `CoursesPage › LevelModal.save` (L537)

Program (group_name), level_name R, billing_type (`session` / `monthly`), total_sessions, age_min, age_max, min qty, sort_order, uf_rate, cf_rate, smf_rate, **student_fee**. A level is one `skus` row; `courses` carries `group_name` and `billing_type`. [C]

2.5 Student admission — `StudentsPage › AddStudentModal` (L3527, `save` L3720)

Field	Required by code	Notes
Mobile (10 digits, `to10Digit`)	**R**	Debounced duplicate lookup (ilike on last 10 digits) across every centre the login can see (RLS-scoped); matches offer "open existing"
Student name	**R**	
Parent email	**R** (`save` rejects blank / no `@`)	The label on the profile tab is not starred; admission blocks without it
Centre	**R**	HO login: fixed to the NLH centre; franchisee login: own centre
Parent name, gender (male/female/other), DOB, camp name, registered_at	O	`registered_at` defaults to today; it is also the invoice date and enrolment start
Channel	O	default `walk-in`; 9 allowed values (`students_channel_check`)
Address block (pincode, city, area, state, country, address)	O	collapsible "Address & extras"
Levels ticked	O	list filtered to the centre's `registered_courses` / `registered_skus` (`courseAccess.deriveFilter`)
Per ticked level: rate (₹)	O	defaults to `skus.student_fee`; edit changes fee_total, invoice line and enrolment `fee_amount`
Per ticked level: kit items given	O	all ticked by default; unticked items are not deducted
Per ticked level: batch (HO only)	O	existing batch of an eligible CI, or "+ create new batch" (CI, name, days, time, individual)
Coupon	O	`CouponField` → `validate_coupon` RPC; not redeemed until first payment
WhatsApp enrolment confirmation	O	ticked by default when a level is chosen

Writes in order: `students` → `enrollments` (+ `fee_amount` correction) → `student_invoices` (+ `stock_ledger` rows) → batch creation / `batch_students` → WhatsApp template → `transactions` mirror. [C]

2.6 Student profile — `StudentDetailModal › save` (L357)

full_name, parent_name, gender, camp_name, dob, registered_at, phone (10-digit enforced), email, address block, channel, fee_total (the "Agreed Fee"). `fee_paid` is read-only (DB trigger). Payment status is read-only (derived). Changing `registered_at` also rewrites `batch_students.assigned_at` for all open batch assignments. [C]

2.7 Receipts — `StudentReceiptModal` (L4558)

Field	Rule
Student	search by name / parent / phone (8 results)
Amount received	R, > 0; blocked when more than balance (`tooMuch`); DB trigger `check_student_payment_total` repeats the rule

Field	Rule
Paid on	date, default today; `paid_at`
Mode	cash, upi, bank_transfer, cheque, card, online (`RECEIPT_MODES`); DB check also allows neft, razorpay, other, opening
Reference	O
Send WhatsApp	default on (new) / off (edit); number re-confirmable
(System) receipt_date	set to today on insert; receipt number assigned by trigger

2.8 Invoices — `InvoiceEditModal` (L4979)

Per course line: course / level (dropdown of all SKUs), amount; per kit item: tick, quantity; add-a-course picker; invoice date; notes. Creation of invoices is not a form: they are produced by Add Course (`addEnrollments`), admission (`AddStudentModal.save`) and Renew cycle (`renewCycle`). [C]

2.9 Renew cycle — `StudentsPage › renewCycle` (L1036)

One dialog for all monthly courses: per course tick, new cycle start (default = due date), fee (default `enrollments.fee_amount`), class days (Mon–Sun chips). [C]

2.10 Attendance — `AttendanceRegister.jsx`

Month grid per batch. Cells: blank (not marked) → P → A → N (no class for the student) → H (batch holiday). Teacher row: blank → P → O (off). Substitute picker when a student is marked present on a teacher-off day. Other attendance inputs exist on the Batches page (`BulkAttendanceModal`) and Instructors page (`AttendanceModal`). [C]

2.11 Instructors — `InstructorsPage`

`AddInstructorModal` (L2121): full name R; phone, email, city, area, state, pincode, joined date; one or more course appointment blocks (course, levels, remuneration mode, per-level rate, trained-by-NLH, training fee/date); caution amount/mode/date; notes. Detail modal tabs: Profile, Caution, Courses, Batches, Payroll. Payroll month inputs: conveyance (monthly, per CI), office salary (monthly, per CI), incentive and misc (per month, `payroll_extras`), "amount paid" override (`payroll_overrides`). [C]

2.12 Coupons — `CouponsPage › CouponModal.save`

code R (upper-cased), description, discount_type (percent/flat), discount_value R, applies_to (orders/students/both), min_amount, max_discount, usage_limit, per_franchisee_limit, valid_from, valid_until, active. [C]

2.13 Logins — `UsersPage › AddUserModal`

email R, name, role R (from `assignableRoles(myRole)`), temporary password R (min 8). Staff logins carry a permission checklist (`PermissionsEditor`). [C]

2.14 External feeds

Feed	Entry point	Payload used
WhatsApp inbound + delivery status	`api/whatsapp-webhook.js` (GET verify, POST)	message text/type/from; status ids → RPC `wa_update_status`; signature verified; broadcast-only auto-reply once per sender per 6 h (`wa_should_autoreply`)
Razorpay	`api/razorpay-webhook.js`, `create-razorpay-order.js`	order payment confirmation (kit orders, not student fees)
BoldSign	`supabase/functions/boldsign-webhook`	agreement signed
File uploads	`MessagesPage.uploadAttachment` → storage bucket `chat-attachments`; receipts/invoices PNG via `captureDocPng` → same bucket	images/documents

---

3. Outputs

3.1 Computed values (student money)

Value	Where computed	Logic
`students.fee_paid`	DB trigger `recompute_student_fee_paid`	Σ `student_payments.amount` for the student
`students.payment_status`	same trigger **and** client `deriveStatus` (StudentsPage L109)	`none` if total = 0; `pending` if paid ≤ 0; `paid` if paid ≥ total; else `partial`. The trigger does not change status for an inactive (closed) student
Balance	client, `fee_total − fee_paid` (never stored)	shown everywhere
Course-wise coverage	`StudentsPage.computeCoverage` (L125)	oldest enrolment first from a pool = payments + implicit discount; used on the Courses tab and receipt screen fallback
Statement lines	`utils/feeStatement.js › buildFeeStatement` (L51)	Rebuilt from `student_fee_events` + `student_invoices` + enrolments: opening lump, later charges, discounts/adjustments
Receipt allocation	`feeStatement.js › allocateReceipt` (L265)	Payments oldest-first against charges net of discounts; per-course rows with month label and "Balance/Part ₹x"
New-receipt preview	`feeStatement.js › receiptPreview` (L217)	Same engine; shows fee / due / this receipt per course
Invoice paid/part/unpaid	`utils/studentLedger.js › invoiceSettlement` (L99)	Payments applied oldest invoice first after charges that have no invoice; discounts counted as settled
Renewal due / state	`utils/studentLifecycle.js › renewalInfo` (L170)	cycle start + 1 month; `overdue` < 0 days, `soon` ≤ 5 days
Cycle target & tally	`studentLifecycle.js › computeCycle` (L112)	class days in window minus holidays minus the student's N days; S/P/A/U/N
Needs-attention reasons	`studentLifecycle.js › attentionReasons` (L190)	renewal overdue/soon, sessions done, certificate pending, balance due
Student bucket (current / attention / completed)	`studentLifecycle.js › studentBucket`	from enrolments
CI payroll month	`InstructorsPage` payroll IIFE (~L1380–1700)	per-session × sessions; per-student on completions × proration; per-student-month × roster × proration; monthly fixed; + office salary + conveyance + incentive ± misc; optional override

3.2 Documents generated

Document	Generator	Delivery
Student invoice (A4)	`components/studentDocs.js › printStudentInvoice` (L300)	print window; stored copy is the `student_invoices` row
Student payment receipt (A5 landscape, one page)	`studentDocs.js › printStudentReceipt` (L366) + `utils/studentReceipts.js`	print; PNG via `captureDocPng` for WhatsApp
Student ledger / statement	`components/StudentLedgerView.jsx` (screen only)	no print/PDF export (the franchisee ledger has CSV + PDF)
Order receipt, combined receipt	`printOrderReceipt` (L436), `printCombinedOrderReceipt` (L485)	print / WhatsApp

Document	Generator	Delivery
Franchisee receipt, enrolment invoice, statement	`printFranchiseeReceipt` (L528), `printFranchiseeEnrollmentInvoice` (L574), `printFranchiseeStatement` (L625)	print / email / WhatsApp
Order invoice	`components/InvoiceView.jsx` (+ `captureInvoicePng`)	on-screen, print, PNG for WhatsApp, email
Franchise agreement / school agreement	`utils/franchiseeAgreement.js`, `agreementPdf.js`, `schoolAgreementPdf.js`	PDF, e-sign
Student certificate	`components/StudentCertModal.jsx` (preview, `printStudentCert`), `public/certificate/Issue Certificate.html`; image rendered server-side by `api/send-cert-whatsapp.js`	print, email (`student_cert`), WhatsApp image, wa.me link
Attendance sheet (student / CI)	`components/AttendanceSheet.jsx`	on-screen, CSV, print
Franchisee cert	`FranchiseeCertModal.jsx`	print / email

3.3 Messages sent

Message	Trigger	Channel / template	Code
Enrolment confirmation	admission (checkbox, ticked by default), Courses & Batches tab strip "Send" (number editable inline, course ticks)	WhatsApp `student_enrolled_v3`	`services/whatsapp.js` › `sendWAStudentEnrolled`; `AddStudentModal.save` L3887; `StudentsPage.sendEnrolmentWA` L488
Payment receipt	New Receipt (checkbox), Receipts register ▸ WhatsApp, Profile ▸ 💬 Receipt, Edit receipt (re-send)	WhatsApp `payment_receipt` (text) or `payment_receipt_v2` (image header)	`sendWAStudentReceipt` → `api/send-payment-whatsapp.js`
Balance reminder	Profile ▸ ⏰ Remind	WhatsApp `balance_reminder`	`sendWAFeeReminder`
Review request	course Actions ▸ Send Google review	WhatsApp `review_request`	`api/send-review-whatsapp.js`
Certificate	Certificate dialog ▸ WhatsApp	WhatsApp image (server renders `Issue Certificate.html`)	`api/send-cert-whatsapp.js`
Certificate	Certificate dialog ▸ Email	Email `student_cert`	`api/send-email.js`
Order invoiced / dispatched	Orders page	WhatsApp `order_invoiced_v3`, `order_dispatched_v3`	`services/whatsapp.js`
Login credentials / reset	franchisee creation, access-request approval, "Resend Login Access"	Email `welcome`, `franchisee_welcome_letter`, Supabase reset email	`services/email.js`
Order emails	confirmation, invoice ready, reminder, verified	Email	`api/send-email.js`
Auto-reply to inbound WhatsApp	any inbound, once per 6 h per sender	WhatsApp text	`api/whatsapp-webhook.js` › `autoReply`

WhatsApp sends from the UI go through `components/WhatsAppSendConfirm.jsx` (number confirm) except: the enrolment-confirmation checkbox at admission, the Courses-tab enrolment strip (number field inline), and the Inbox free-text reply. [C]

3.4 Reports / registers

Students page: summary cards (current / completed / fees charged / fees received / balance due), tabs Current · Needs attention · Completed · All · **Attendance** · **Invoices** · **Receipts**, CSV export (`exportCSV` L5740). Dashboard KPIs and charts. Accounting overview, P&L, journal, splits. Franchisee ledger (CSV/PDF). Payroll per CI per month. [C]

---

4. Data model

Source: live `information_schema`, `pg_constraint`, `pg_trigger` (2026-10-12) and `supabase/migrations/` (160 files). Only tables that carry the enrolment / fee / attendance / certification flows are listed in full. [C]

4.1 Entity map

```
users ──(franchisee_id)──────────────┐
  └─(student_id)──► students ◄────────┤  students.franchisee_id → franchisees
                       │              │
 franchisees ◄─parent_id─ franchisees (self-reference tree)
                       │
students 1─* enrollments *─1 skus *─1 courses          (enrollments.student_id, .sku_id; skus.course_id)
students 1─* student_invoices (items jsonb; enrollment_id optional)
students 1─* student_payments  (receipt_no, receipt_date, paid_at)
students 1─* student_fee_events (audit log, written by triggers)
enrollments 1─* batch_students *─1 batches *─1 instructors  (assignment history: assigned_at / removed_at)
batches 1─* batch_sessions 1─* session_attendance *─1 enrollments
enrollments 1─* student_no_class (enrollment_id, class_date)
instructors 1─* instructor_courses *─1 skus   (appointments + remuneration)
instructors 1─* instructor_attendance (P/O per date), payroll_overrides, payroll_extras
skus 1─* kit_items *─1 inventory_items;  inventory_items 1─* stock_ledger (ref_type 'enrollment' for kit issued)
coupons 1─* coupon_redemptions (context 'order' | 'student')
transactions / transaction_items / transaction_payments  (mirror of money tables; not read by UI)
```

4.2 Key tables

`students` — `id`, `full_name`, `parent_name`, `phone` (10-digit by convention), `email`, `dob`, `gender` (male/female/other), address block (`address, area, city, state, pincode, country`), `channel` (9 values), `camp_name`, `registered_at`, `franchisee_id`, `is_active`, `closed_at`, `close_reason`, **money:** `fee_total` (stored running number), `fee_paid` (trigger-maintained), `payment_status` (free text, no check), `other_charges`, `waived_amount`, `discount_amount`, `coupon_id`, `coupon_code`, `payment_mode` (not written by current UI), `notes`. *Derived:* balance = `fee_total − fee_paid` (never stored).

`enrollments` — `student_id`, `sku_id`, `franchisee_id`, `enrolled_at`, **fee:** `fee_amount` (what this course is billed at; defaulted from `skus.student_fee` by trigger `default_enrollment_fee`), `list_price`, `waived`; **status:** `status` (active/completed/dropped), `completed_at`; **monthly cycle:** `sessions_per_week`, `sessions_per_cycle`, `cycle_started_at`, `cycle_days`; **certification:** `cert_status` (school flow), `marks_*`, `cert_emailed_at`, `cert_wa_sent_at`, `cert_wa_message_id`, `cert_wa_status`, `cert_issued_at/by/note`, `cert_reviewed_*`, `cert_reject_note`; legacy: `fee_paid`, `payment_ref`, `payment_mode`, `next_level_unlocked`, `notes`.

`student_invoices` — `invoice_no` (`SINV-YYYY-NNNN`, unique, trigger), `invoice_date`, `items` (jsonb lines, see below), `subtotal`, `discount`, `coupon_code`, `total`, `amount_paid`, `status` (`unpaid/part/paid` by convention), `notes`, `created_by`, `enrollment_id` (set only when the invoice has exactly one course). *Line shape:* `{kind:'course'|'kit', sku_id, enrollment_id, name, qty, rate, amount, cycle?:'renewal'|'next_level', period_start?, period_label?, original_amount?, edited_at?}`.

`student_payments` — `amount` (> 0), `mode`, `reference`, `paid_at` (money received), `receipt_date` (issue date), `receipt_no` (`ST-<centre>-<year>-NNNN`, trigger), `receipt_no_previous`, `note`, `franchisee_id`.

`student_fee_events` — append-only log: `field` ∈ {`fee_total`, `fee_amount`, `waived`, `status`, `other_charges`, `waived_amount`}, `old_value`, `new_value`, `delta`, `actor` (JWT email), `enrollment_id`, `at`. Written only by triggers `trg_log_student_fee` and `trg_log_enrollment_fee`. Used to rebuild statements (`feeStatement.buildFeeStatement`).

`batches` — `name`, `instructor_id` (required), `franchisee_id`, `sku_id` (nullable — 12 of 30 batches have none), `schedule_days` (text like `Mon, Tue`; 1 of 30 empty), `schedule_time`, `start_date`, `is_individual`, `is_active`, `sessions_done` (counter, maintained by client), `notes`.

`batch_students` — `batch_id`, `enrollment_id`, `assigned_at`, `removed_at`, `assigned_by`. One enrolment may have several rows for the same batch (re-assignment history); unique on `(batch_id, enrollment_id, assigned_at)`.

`batch_sessions` — `batch_id`, `session_date`, `session_number`, `instructor_id` (who took it), `is_substitute`, `is_holiday`, `notes`. `session_attendance` — `session_id`, `enrollment_id`, `student_id`, `attended` (bool), unique `(session_id, enrollment_id)`. *Absence of a row = not marked.*

`student_no_class` — `enrollment_id`, `class_date`, `batch_id`; unique `(enrollment_id, class_date)`. `instructor_attendance` — `instructor_id`, `att_date`, `status` P/O.

`instructors` — profile, address, `status`, caution (`caution_amount/mode/status/settled_*`), `conveyance_monthly`, `office_salary_monthly`. `instructor_courses` — `sku_id`, `remuneration_mode` (per_session / per_student / per_student_month / monthly), `remuneration_rate`, `appointed_at`, `removed_at`, `status`, `trained_by_nlh`, `training_fee`, `training_date`; unique `(instructor_id, sku_id, appointed_at)`.

`franchisees` — `tier` (SMF/CF/UF/NLH/SCHOOL), `parent_id`, `business_name`, `owner_name`, contact, address block, `status`, `registered_courses[]`, `registered_skus[]`, `enrollment_fee`, `fee_paid` (trigger from `franchisee_payments`), `payment_status`, `contract_start/end`, `valid_till`, `renewal_fee`, `centre_code` (unique; trigger `assign_centre_code`), `enrollment_invoice_no` (`FEI-HO-NNNN`), `cert_emailed_at`.

`users` — `email` (unique), `role`, `franchisee_id`, `student_id`, `is_active`, `permissions` (jsonb, staff only). Row id is aligned to the auth user id by trigger `align_user_id_to_auth`.

`courses` / `skus` — `courses(group_name, billing_type session|monthly)`; `skus(level_name, total_sessions, uf_rate, cf_rate, smf_rate, student_fee, sort_order, sku_type, supply_category)`.

`coupons` / `coupon_redemptions` — see §2.12; redemptions unique per `(coupon_id, context, ref_id)`.

4.3 Derived fields — where each one lives

Derived value	Stored?	Source of truth
Student balance	no	`fee_total − fee_paid`
Student `fee_paid`	yes	Σ `student_payments` (trigger)
Student `payment_status`	yes	trigger **and** client `deriveStatus`
Course-wise paid/due	no	`computeCoverage`, `buildFeeStatement` + `allocateReceipt` (two engines)
Invoice paid/part/unpaid	yes (stale) **and** computed	stored `status/amount_paid` vs `invoiceSettlement`
Receipt line "Balance ₹x / Part ₹x"	no	`allocateReceipt` at print time
Renewal due date	no	`cycle_started_at + 1 month`
Sessions held / attended	`batches.sessions_done` (counter) and live counts	client-maintained counter vs rows
Certificate pending	no	`certPending` (completed and none of sent / emailed / issued)
Student bucket / attention	no	`studentLifecycle`
Franchisee `fee_paid`	yes	trigger `sync_franchisee_fee_paid`
Order totals	yes	trigger `recalc_order_totals`

4.4 Numbering series

Series	Format	Generated by
Student invoice	`SINV-YYYY-NNNN` (year from `now()`, one global sequence `student_invoice_seq`)	trigger `assign_student_invoice_no`
Student receipt	`ST-<centre_code>-<year>-NNNN` (counter row per series/centre/year in `receipt_counters`; year from `paid_at`)	trigger `assign_student_receipt_no` → `next_receipt_no`
Franchisee fee receipt / enrolment invoice	`FR-<centre>-<year>-NNNN` (counter series `FR`), `FEI-HO-NNNN`	triggers `assign_franchisee_receipt_no`, `assign_franchisee_enrollment_invoice_no`

Series	Format	Generated by
Order ref / proforma / invoice / order receipt	`order_ref`, `proforma_no`, `invoice_no`, `OR-<centre>-<year>-NNNN`	triggers `generate_order_ref`, `generate_proforma_no`, `generate_invoice_no`, `assign_order_receipt_no`
Centre code	`HO`, or city stem + 2 digits (`NGP02`)	trigger `assign_centre_code`

4.5 Access control (RLS)

Tables read in this audit use the same pattern: `nlh_is_admin()` OR the row's centre is in `nlh_accessible_franchisee_ids()` (recursive descendants of the login's franchisee) — plus per-key `staff_perm_*` policies using `nlh_staff_can('{students.view}'…)`. Enrolment-linked tables (`session_attendance`, `student_no_class`) test access through the parent enrolment's `franchisee_id`. [C]

---

5. Business rules (as implemented)

Each rule: statement → where it lives. "Client" = enforced only in the browser; "DB" = trigger/constraint.

#	Rule	Where	Layer
R1	One SMF per state (per country abroad); one CF per city; UFs unlimited	`FranchiseesPage.AddFranchiseeModal.save`, `OnboardingPage.submitFranchisee`	Client only
R2	A centre can enrol students only in levels it is registered for	`utils/courseAccess.js › deriveFilter / filterSkusForFranchisee`; `AddStudentModal.buildGroups`	Client (list filtering)
R3	SMF and CF get all active courses on creation; UF starts with none	`AddFranchiseeModal.save`, `AccessRequestsPage.handleApprove`	Client
R4	Franchise contract = 3 years from creation (−1 day)	same two functions	Client
R5	`fee_total` is a stored running number moved by: admission (net of coupon), Add Course (+net), Renew cycle (+Σ fees), invoice edit (± difference), discontinue (−due), close (−Σ due), reopen (+waived), other charges (±), coupon discount (−), manual "Agreed Fee" edit	`AddStudentModal.save`; `addEnrollments` L1348; `renewCycle` L1036; `InvoiceEditModal.save`; `discontinueCourse` L1509; `closeStudentAccount` L1565; `reopenStudentAccount` L1607; `saveOtherCharges` L469; `applyFeeDiscount` L1271; `saveFeeOnly` L1260	Client; every change logged by DB trigger `log_student_fee_change`
R6	Editing a course's own fee (`saveCourseFee`) does **not** move `fee_total`; the gap against `fee_total` is read as the discount	`StudentsPage.saveCourseFee` L447; `computeCoverage`	Client
R7	`enrollments.fee_amount` and `list_price` default to the SKU's `student_fee` at insert	trigger `default_enrollment_fee`	DB
R8	Total payments may not exceed `fee_total` when `fee_total > 0` (checked on insert/update of a payment only)	trigger `check_student_payment_total`; UI `StudentReceiptModal.tooMuch`	DB + Client
R9	`fee_paid` = Σ payments; `payment_status`: none (total 0) / pending (paid ≤ 0) / paid (paid ≥ total) / partial; a closed student's status is left alone by the trigger	`recompute_student_fee_paid`; `deriveStatus` L109	DB + Client
R10	Payments are held against the **student**, not a course; allocation to courses is oldest charge first, after discounts	`feeStatement.allocateReceipt`, `receiptPreview`; `studentLedger.invoiceSettlement`; `computeCoverage`	Client (three implementations)

#	Rule	Where	Layer
R11	Receipt number series is per centre and year; year follows `paid_at`; receipt "Date" shown = `receipt_date`, "Paid on" = `paid_at`	`assign_student_receipt_no`, `next_receipt_no`; `studentDocs.printStudentReceipt`	DB
R12	Invoice number is one global yearly-labelled sequence	`assign_student_invoice_no`	DB
R13	A coupon is validated at entry, stored on the student, and **redeemed (counted) only when the first payment is saved**	`validate_coupon` RPC; `StudentReceiptModal.save` (payments.length === 0 → `redeem_coupon`)	DB RPC + Client trigger point
R14	Coupon discount = percent × amount or flat, capped by `max_discount` and the amount; limits: `min_amount`, `usage_limit`, `per_franchisee_limit`, validity window, `applies_to`	`_coupon_discount`, `validate_coupon`	DB
R15	Monthly cycle: starts at `cycle_started_at` (else `enrolled_at`), runs to the same date next month; target = count of the student's class days in the window, minus batch holidays, minus the student's own N days; Saturday not among class days is "revision" and only makes up marked absences; Sunday counted only if it is a class day	`studentLifecycle.computeCycle` L112	Client
R16	Renewal: sets new `cycle_started_at` + `cycle_days`, adds the fee to `fee_total` once, raises **one** invoice (a line per course, each tagged with its month)	`StudentsPage.renewCycle` L1036	Client
R17	Renewal state: `overdue` if due date passed, `soon` within 5 days (`RENEW_SOON_DAYS`)	`studentLifecycle.renewalInfo`	Client
R18	Monthly-billed vs per-course-fee: `courses.billing_type`; only `monthly` has cycles/renewal; `session` courses finish by attended sessions vs `skus.total_sessions`	`isMonthlyActive`; `attentionReasons`	Client
R19	Completion sets `completed_at` (default = last attended class date) and `status='completed'`; school-tier enrolments submit marks and wait for HO certify/reject	`markCourseComplete` L916; `certifyEnrollment` L990	Client
R20	Certificate is "pending" while completed and none of `cert_wa_sent_at`, `cert_emailed_at`, `cert_issued_at` is set; the certificate carries the latest completion date among the courses on it	`studentLifecycle.certPending`; `StudentCertModal.certDateYMD`; `api/send-cert-whatsapp.js`; `api/send-email.js › buildStudentCert`	Client + server
R21	Discontinue a course: status `dropped`, unpaid part of its fee waived (reduces `fee_amount` and `fee_total`, stored in `waived`), no certificate; reversible	`discontinueCourse`, `restoreCourse`	Client
R22	Close account: every unfinished course discontinued, outstanding balance waived, student `is_active=false`, `closed_at`, `close_reason`; payments and receipts kept; reopen restores waived amounts	`closeStudentAccount`, `reopenStudentAccount`	Client
R23	Delete student allowed only when no payment exists; cascades enrolments, batch rows, invoices, payments	`deleteStudent` L1636; FK `ON DELETE CASCADE`	Client check + DB cascade
R24	Batch membership on a date = any `batch_students` period containing the date, enrolment not completed before it	`AttendanceRegister.memberOn`, `BatchesPage.studentsForDate`	Client

#	Rule	Where	Layer
R25	Re-assigning to a batch re-activates the existing row instead of inserting; switching batch closes the old row	`assignToBatch` L842; `BatchesPage.RosterModal.assign`	Client
R26	Attendance: one session per batch per date; one row per enrolment; **no row = not marked, not absent**; holiday is per batch/date; N = no class for that student that date	`AttendanceRegister` ; `computeCycle`	Client
R27	A teacher "O" (off) day is per instructor per date; a student marked present that day needs a substitute, stored as `batch_sessions.instructor_id` + `is_substitute`	`AttendanceRegister.stepTeacher / withSubstitute / saveBatch`	Client
R28	Kit issued to a student deducts HO stock (`stock_ledger` issue rows referencing the enrolment) only for items ticked as given	`addEnrollments` , `AddStudentModal.save` , `toggleKitGiven` L1114, `InvoiceEditModal.save`	Client
R29	CI pay per month = per-session × sessions taught, or per-student × completions × proration (sessions taught ÷ batch sessions), or per-student-month × roster × proration, or monthly fixed; plus office salary, conveyance, incentive ± misc; an "amount paid" override replaces the total	`InstructorsPage` payroll block	Client
R30	WhatsApp numbers are normalised to `91` + 10 digits; student phone stored as bare 10 digits	`services/whatsapp.js › toWAPhone` ; `StudentsPage.to10Digit`	Client

6. UI inventory

6.1 Navigation

Sidebar groups (`Sidebar.jsx`): **Operations** (Dashboard, Franchisees, Orders, Students, Instructors, Batches, WhatsApp Inbox, Franchisee chat), **Accounting** (HO Accounting), **Settings** (Courses & SKUs, Coupons, Inventory & Kits, Kit prices, Price log, Email log, Manage logins, Access requests). Franchisee roles see a reduced list with renamed items ("My students", "My orders", "My Account", "Head Office"). [C]

6.2 Students module — screens and modals

Screen / modal	Purpose	Main actions	Reached by
Students list	Roster with cards and tabs	search, centre filter, sort, Closed toggle, CSV export, **+ Enrol Student**, open student	Sidebar ▸ Students
Tab: Current / Needs attention / Completed / All	Lifecycle views	row click opens inline detail; "Mark certificates issued (n)" bulk	list tabs
Tab: **Attendance**	Monthly P/A/N/H register per batch + teacher row	step cells, ✓ everyone present, Save per batch, pending-class chips, per-student sheet search	list tab
Tab: **Invoices**	Register of all student invoices	Print, Edit	list tab
Tab: **Receipts**	Register of all payments	Print, WhatsApp, Edit, **+ New Receipt**	list tab
Student detail (inline) — Profile	Personal data, fee tracking, payment history, invoice history, fee-change history	edit & Save, Agreed Fee Save, ⏰ Remind, discount coupon, receipt Print / 💬 / Edit / 🗑, invoice Print / Edit / 🗑	row click
Student detail — Courses & Batches	Per-course cards, Needs-attention panel, enrolment WhatsApp strip, + Add other charges, + Add Course	🗓 Renew cycle, Attendance sheet, per-course **Actions** menu (Complete, Certificate, Mark certificate issued, Google review, Assign / Change batch, Remove from batch, Change course/level, Discontinue/Restore, Remove course), fee pencil, kit badge panel	tab

Screen / modal	Purpose	Main actions	Reached by
Student detail — Accounts	Ledger: charges, discounts, payments with running balance	Invoice / Receipt print, ✎ Edit	tab
Footer	account actions	Delete Student, Close / Withdraw (Reopen), Close	detail footer
New Student modal	Admission + invoice entry	phone-first duplicate check, fields, invoice table, Add Student	+ Enrol Student
New / Edit Receipt modal	Record or correct a payment	amount, paid on, mode, reference, WhatsApp	+ New Receipt, Edit
Edit invoice modal	Change lines, kit, add a course	amount, level, kit tick/qty, add course, date, notes	Edit (3 places)
Renew cycle modal	Multi-course renewal	tick, start, fee, days	🗓 Renew cycle
Change course / level modal	Swap SKU in place	level, start date, kit items	Actions ▸ Change course / level
Complete course modal	Completion date (+ marks for school)	date, marks	Actions ▸ Complete
Certificate modal	Preview, print, email, WhatsApp	course ticks, send	Actions ▸ Certificate
Attendance sheet modal	Month grid for one student (or CI)	prev/next, CSV, print	button / register search
MarkCertsIssuedModal	Bulk "already handed over"	cut-off date, exclusions	Needs attention / Completed tabs
WhatsAppSendConfirm	Confirm number before any send	edit number, send	every WhatsApp button

6.3 Other modules (summary)

Module	Screens
Franchisees	List with tier/status filters; detail (10 tabs); Add Franchisee; Add School; School rates; Record payment; resend login; change email; agreement generate/sign/send; certificate print/email; ledger; CIs and commission (billing roles)
Instructors	List; detail (Profile, Caution, Courses, Batches, Payroll); Add Instructor (with appointment blocks); attendance modal per batch
Batches	Cards with roster, filters; New batch picker → Add batch; Edit batch (days/time); Change instructor; Roster modal; Bulk attendance; Session history (edit/delete sessions)
Orders	New order; proforma → invoice; record payment; dispatch; returns/credit notes; Razorpay online pay; reminders
Accounting	Overview, Accounts, Expenses, Journal, Splits, Vendors, Banks
Courses & SKUs	Program/level matrix; level modal; rename program
Logins / Requests	Add user; edit role & staff permissions; block; delete; invite; approve/reject requests (credentials dialog)

6.4 Step counts for common tasks

Counts are clicks/screens on the shortest current path as a HO admin; typing is counted as one entry per field. [C — derived from the components above]

Task	Path	Clicks	Entries	Separate flows
Enrol a new student, no payment	Students ▸ + Enrol Student ▸ (mobile) ▸ tick level(s) ▸ Add Student	4 + one per level (+1–2 per level for batch)	mobile, name, parent email (R) + optional	1
...and take the first payment	Receipts tab ▸ + New Receipt ▸ search student ▸ pick ▸ (amount prefilled) ▸ Save Receipt	+5	amount (prefilled to balance), mode	2 (admission and receipt are separate)
Renew a monthly course	Students ▸ student row ▸ Courses & Batches ▸ 🗓️ Renew cycle ▸ (check rows) ▸ Renew N courses	5	per row: date (prefilled), fee (prefilled), days (prefilled)	1; **collecting the fee is a second flow** (+5)
Correct an invoice amount	Students ▸ Invoices tab ▸ Edit ▸ amount ▸ Save	4	amount	1
Correct the agreed total	Students ▸ row ▸ Profile ▸ Agreed Fee ▸ Save (fee)	4	amount	1 (does not change invoice or course fee)
Change a course's fee	Students ▸ row ▸ Courses & Batches ▸ fee pencil ▸ value ▸ save	5	value	1 (does not change agreed total or invoice)
Correct a receipt	Receipts tab ▸ Edit ▸ fields ▸ (tick re-send) ▸ Save Changes	4–5	fields	1
Re-send a receipt on WhatsApp	Receipts tab ▸ WhatsApp ▸ (confirm number) ▸ Send	3	—	1
Record attendance for one class day	Students ▸ Attendance tab ▸ find batch ▸ click each student's cell for the date ▸ Save changes	3 + one per student	—	1 (alternatives: Batches ▸ Mark attendance; Instructors ▸ CI ▸ Batches ▸ Attendance)
Complete a course and send the certificate	row ▸ Courses ▸ Actions ▸ Complete ▸ (date) ▸ Save ▸ Actions ▸ Certificate ▸ WhatsApp ▸ confirm ▸ Send	9	date	2 dialogs; Google review request is a third (Actions ▸ review ▸ confirm ▸ send)
Move a student to the next level in the same batch	row ▸ Courses ▸ + Add Course ▸ tick level ▸ (batch preselected) ▸ Enroll	5	rate (prefilled)	1; creates a new enrolment, invoice, kit issue
Withdraw a student	row ▸ Close / Withdraw ▸ confirm ▸ reason prompt	4	reason	1
Onboard a franchisee	Franchisees ▸ + Add ▸ fields ▸ Create (or approve an access request)	3 + parent pick	owner, email, tier, parent	1; then fee, courses, agreement are set in detail tabs (separate saves)

---

7. Friction audit

Findings are recorded without remedies. Each is tied to a flow and to code. Severity is the audit's observation of consequence, not a priority: **H** = affects money or data correctness; **M** = repeated effort or inconsistent behaviour; **L** = cosmetic or latent.

7.1 Money: one fact, several places

#	Finding	Flow	Evidence	Sev
F1	The student's "what is owed" can be changed from five screens with different side effects: Agreed Fee field, invoice edit, course-fee pencil (changes the enrolment only), Renew cycle, Add other charges / coupon. Only some also change the invoice, enrolment fee or statement	Fee correction	`saveFeeOnly` L1260, `saveCourseFee` L447, `InvoiceEditModal.save`, `renewCycle`, `saveOtherCharges`, `applyFeeDiscount`	H

#	Finding	Flow	Evidence	Sev
F2	In the live data, 18 of 37 students have `fee_total ≠ Σ enrolment fee_amount + other_charges` (some intentional: discounts/waivers; the system cannot tell which)	Fee correction / reporting	live query 2026-10-12; `computeCoverage` treats the gap as "discount"	H
F3	Three engines compute "what a payment covered": `computeCoverage` (enrolment fees), `buildFeeStatement` + `allocateReceipt` (event log + invoices), `invoiceSettlement` (invoice-level). They can disagree for the same student	Receipts, ledger, course cards	`StudentsPage.computeCoverage`, `feeStatement.js`, `studentLedger.js`	H
F4	Invoice paid/part/unpaid exists twice: stored `student_invoices.status/amount_paid` (written once at creation) and a derived view. The Profile ▸ Invoice history list and `handlePrintInvoice` (no settlement passed) use the stored value; Accounts tab and Invoices register use the derived one	Invoices	`StudentsPage` L1966 vs `StudentLedgerView`, `StudentInvoicesRegister`	H
F5	`payment_status` is derived by a DB trigger, by client `deriveStatus`, and set to `waived` by the close-account handler; vocabularies differ across layers (`pending/partial` for students, `unpaid/part/paid` for invoices, `part_paid` in `transactions`)	Status display	`recompute_student_fee_paid`, `deriveStatus` L109, `closeStudentAccount` L1565, `mirrorStudentToTransaction` `statusMap`, `invoiceSettlement`	M
F6	The `transactions` mirror is not updated by Renew cycle, Change level, Invoice edit (both entry points) or Receipt edit; it is only updated on create / add course / discount / discontinue / close / reopen / other charges and on receipt create/delete. Nothing reads it, so drift is silent	All money edits	grep of `mirrorStudentToTransaction` (10 call sites) vs `renewCycle`, `saveChangeLevel`, `InvoiceEditModal.save`, `StudentReceiptModal.saveEdit`	M
F7	The payment cap is checked only when a payment is written; lowering `fee_total` below what is already paid (Agreed Fee, invoice edit, discontinue) is allowed and leaves balance negative / `paid`	Fee correction	`check_student_payment_total` fires on `student_payments` only	M
F8	Receipt "Date" vs "Paid on": the numbering year follows `paid_at`, so a back-dated payment takes a number in an earlier year's series; the edit screen cannot change `receipt_date`; the register sorts by `receipt_date` but labels the payment date as a small sub-line	Receipts	`assign_student_receipt_no`; `StudentReceiptModal.saveEdit`; `StudentReceiptsRegister`	M
F9	Net effect of "Edit invoice amount": the Fee Total moves by the delta and the statement shows an adjustment line, but the course's own `fee_amount` and future renewal default do not move	Invoice edit	`InvoiceEditModal.save`	M

7.2 Enrolment, batches, renewal

#	Finding	Flow	Evidence	Sev
F10	Batches can be created in five places, each with its own copy of the form: Batches ▸ New, Instructors ▸ CI ▸ Batches, Student ▸ course card panel, Add Course, New Student	Batch set-up	`BatchesPage.AddBatchModal`, `InstructorsPage.saveBatch` L468, `StudentsPage.createBatchAndAssign` L887, `addEnrollments` L1348, `AddStudentModal.save`	M
F11	A student can be put in a batch from five places (card panel, Add Course, New Student, Batches ▸ Roster, Instructors ▸ CI ▸ Batches), with different candidate lists: card panel by SKU/CI appointment, roster by CI appointment, others by centre. `batches.sku_id` is empty on 12 of 30 batches, which hid them from SKU-filtered lists until the continuing-batch and CI-appointment lookups were added	Batch assignment	`openBatchPanel` L786, `loadAddBatchData` L1297, `BatchesPage.RosterModal`, `InstructorsPage.openAddStudentPanel` L509; live: 12 batches without `sku_id`	M
F12	Batch joining date has three meanings: set once and "never re-edited" (card panel), editable (`RosterModal.saveJoinDate`), and overwritten from `registered_at` on profile save	Batch roster	comment above `assignToBatch` L842; `BatchesPage` L1315; `StudentDetailModal.save` L357	M

#	Finding	Flow	Evidence	Sev
F13	Attendance can be recorded in three places with different defaults: Batches ▸ bulk modal and Instructors ▸ attendance modal default everyone **present** (`attended !== false`); the Students register defaults **not marked**	Attendance	`BatchesPage.BulkAttendanceModal.saveAll` L198; `InstructorsPage.AttendanceModal.save` L2006; `AttendanceRegister`	H
F14	A course can be marked complete from two places: Students (with the school marks / HO review branch) and Batches ▸ roster (no marks branch, no school gate)	Completion	`StudentsPage.markCourseComplete` L916; `BatchesPage.markComplete` L1601	M
F15	"Next level" can be done two ways with different outcomes: Add Course (new enrolment, new invoice, new kit, batch pick) and Change course/level (swaps SKU in place, keeps batch and attendance, no invoice). Session-billed courses keep counting attendance across the swap	Next level	`addEnrollments` , `saveChangeLevel` L1177, `sessionCounts` (all attended rows per enrolment)	M
F16	Renewal is manual and per student; the only cross-student view is the Needs attention tab (reason chip only). Fee, class days and start date are prefilled but re-confirmed each month; collecting the money is a separate flow (Receipts)	Renewal	`renewCycle` ; `attentionReasons` ; step counts §6.4	M
F17	`batches.sessions_done` is a counter maintained by client code (`adjustSessions ±1` , recomputed on save), while sessions/attendance are rows; both are read	Batches	`BatchesPage.adjustSessions` L1520, `AttendanceRegister.saveBatch` , `InstructorsPage.adjustSessions` L502	L
F18	Batches with no `schedule_days` (and 12 without SKU) disable missed-class detection; the register falls back to Mon–Sat	Attendance	`AttendanceRegister` (`dowSet(...).size === 0`); live: 1 batch without days	L
F19	A student's "no class" day (N) is stored in its own table and read only by the Students register, the student sheet and the cycle maths; the Batches and Instructors attendance modals do not read it and list every rostered student (default present)	Attendance	`student_no_class` read only in `AttendanceRegister` , `AttendanceSheet` , `StudentsPage.loadNoClassMap`	M

7.3 Re-entry and inferable values

#	Finding	Flow	Evidence	Sev
F20	The same address block (country, state, city, area, pincode, address) is a separate form in: access request, onboarding (two forms), Add Franchisee (`LocationFields`), Franchisee detail (inlined copy), Add School, Add Instructor, New Student, Student profile	Data entry	`AccessRequestsPage` , `OnboardingPage` , `FranchiseesPage.LocationFields` L104 + L412, `AddInstructorModal` , `AddStudentModal` , `StudentDetailModal`	M
F21	City / state / country (optional fields) are typed per student although the centre is already chosen; parent name, phone and email are re-typed when a parent has a second child (the phone lookup offers "open existing", not "copy parent")	Admission	`AddStudentModal` (phone lookup `phoneMatches`)	M
F22	The first receipt is a second flow after admission, though admission already knows the fee: no "record payment now" on the admission screen	Admission → payment	§6.4	M
F23	Receipt amount is prefilled with the full balance; the system knows renewal fee, last payment mode and the parent's phone yet asks for mode and WhatsApp number each time (number is prefilled, mode defaults to Cash)	Receipt	`StudentReceiptModal`	L

#	Finding	Flow	Evidence	Sev
F24	The admission form requires parent e-mail even though the student has no login and e-mail is optional on the profile; the label on the profile tab carries no asterisk	Admission	`AddStudentModal.save` vs `StudentDetailModal` form	L
F25	Enrolment confirmation WhatsApp is sent at admission with the number typed once, without the shared number-confirm step used by every other send	Admission	`AddStudentModal.save` L3887 vs `WhatsAppSendConfirm` usage	L
F26	Kit: issued by default for every level at admission/Add Course; confirming "what was actually handed over" is a later per-item action on the card; three separate kit UIs (invoice line, card badge panel, Change level)	Kit	`loadAddKit`, `toggleKitGiven` L1114, `saveChangeLevel`	M
F27	Certificate path is 9–10 clicks across two dialogs; the Google review request is a third; "Already issued" is a fourth path with no document	Completion	§6.4; `markCertIssued` L978	M
F28	Course price shown in three places (SKU `student_fee`, enrolment `list_price` / `fee_amount`, invoice line `rate`) with no live link after creation	Pricing	`default_enrollment_fee`; `InvoiceEditModal`	M

7.4 Access, security and operations

#	Finding	Flow	Evidence	Sev
F29	Every created login gets the fixed temporary password `NLH@123` (`genTempPass()` returns a constant; approval of an access request hard-codes the same) and the password is e-mailed in clear	Onboarding	`FranchiseesPage.genTempPass` L330, `StudentsPage.genTempPass` L189, `AccessRequestsPage.handleApprove`, `services/email.sendWelcomeEmail`	H
F30	Territory uniqueness (R1) is enforced only in the browser	Franchisee onboarding	no DB constraint for tier/state/city	M
F31	Many multi-step writes are sequences of independent client calls without a transaction (admission: student → enrolments → invoice → stock → batches; renewal; close account loops per enrolment). A failure part-way leaves partial state; only a few places roll back (`AccessRequestsPage.handleApprove` deletes the franchisee when login creation fails)	All write flows	`AddStudentModal.save`, `addEnrollments`, `closeStudentAccount`, `InvoiceEditModal.save`	H
F32	The student list query has no paging or limit: a 1,000-row API cap would silently truncate the roster (37 students today). Attendance and invoices are paged through `fetchAllRows`; students are not	Students list	`StudentsPage` list `useEffect` (`select('*, franchisees…, enrollments…')`)	M
F33	Dead or legacy columns remain on live tables: `enrollments.fee_paid`, `payment_ref`, `payment_mode`, `next_level_unlocked`; `students.payment_mode` (read into form state, never written)	Data model	grep over `src` shows no write path	L
F34	The Franchisee-chat module, WhatsApp Inbox and email log are separate message stores (`messages`, `whatsapp_messages`, `email_log`) with no per-student/parent thread view; a send shows in the Inbox only because `logOutbound` is called	Communication	`MessagesPage`, `WhatsAppInboxPage`, `services/whatsapp.logOutbound`	L
F35	The WhatsApp template docs are out of date relative to code: `docs/whatsapp-templates.md` lists `payment_receipt_v2` as pending while `WA_TEMPLATES.paymentReceiptImage` is set	Documentation	`src/services/whatsapp.js` L18–23 vs `docs/whatsapp-templates.md`	L
F36	Navigation is internal state, not URLs: no deep links, browser back leaves the app, an open student detail is lost on refresh; nested modals (detail → receipt → WhatsApp confirm) stack	All	`AppShell.currentPage`; `StudentsPage` `selected` state	M

#	Finding	Flow	Evidence	Sev
F37	`StudentsPage.jsx` is 6,155 lines holding ~25 screens/modals and its own money engines; `OrdersPage.jsx` 4,452; behaviour is duplicated (two receipts flows, two invoice editors before consolidation, three attendance writers)	Maintainability	file sizes	M

---

8. Open questions and assumptions

Marked **A** = the conservative assumption this audit would take if no answer is given.

#	Question	Why it matters	Assumption (A)
Q1	Which number is authoritative for a student's fee: `students.fee_total`, Σ `enrollments.fee_amount`, or Σ invoices?	F1–F3	**A:** `fee_total` is authoritative (payments are checked against it), invoices and enrolment fees are supporting detail
Q2	Is a gap between Σ course fees and `fee_total` always a "discount"?	F2, `computeCoverage`	**A:** yes for display; not for reporting until confirmed
Q3	Should a first payment be capturable on the admission screen?	F22	**A:** unknown — policy
Q4	Attendance default: present-by-default (Batches/Instructors) or not-marked (Students register)?	F13, unmarked counted in S	**A:** not-marked is correct because no row means unmarked in every report
Q5	May franchisee logins (UF/CF/SMF) manage batches and instructors? Code gates batch assignment/creation on the head-office login (`admin` checks in Add Course / New Student)	F10, F11	**A:** head office only, as the code comments state
Q6	Which date should a receipt number's year follow: receipt date or payment date?	F8	**A:** payment date (current)
Q7	Are school-tier marks / HO review a live process? (`cert_status` flow exists; only school enrolments use it)	F14	**A:** live for SCHOOL tier only
Q8	Is the `transactions` mirror intended to become the system of record?	F6	**A:** yes (per `docs/transaction-model-migration-plan.md`), not yet
Q9	Are `enrollments.fee_paid`, `payment_ref`, `payment_mode`, `next_level_unlocked`, `students.payment_mode` retired?	F33	**A:** retired; safe to ignore
Q10	Should a student be allowed to hold more than one enrolment in the same level (re-take)? The DB has no unique `(student_id, sku_id)`	enrolment	**A:** no; the UI blocks duplicates in Change level and invoice edit only
Q11	Is "payments held against the student, allocated oldest first" the intended business rule rather than per-course payments?	F3, receipts	**A:** yes (as built and explained to users)
Q12	Stock for kits is always HO stock (`location_type 'ho'`) even for franchisee-run centres — intended?	R28	**A:** yes
Q13	Who may delete payments/invoices? Profile allows delete for admins; register has Edit only	F-series	**A:** admins only
Q14	What should happen to `batches.sessions_done` for a holiday later converted to a class?	F17	**A:** recomputed from rows on every register save (as built)
Q15	Student-role logins ("My courses"): is there a student-facing flow beyond onboarding? Only a Dashboard and `CoursesPage` view are mapped; not audited in depth	scope	**A:** read-only
Q16	CI payroll proration (sessions taught ÷ all batch sessions) and the "amount paid" override: is override a payment record or only a note?	payroll	**A:** note of agreed figure; no payment ledger
Q17	Parent e-mail: required at admission but optional elsewhere — intentional?	F24	**A:** intentional for receipts/certificates by e-mail

Reconciliation of the reference screenshots (labelled [S])

Description in the brief	What the code shows
Students list tabs "Current, Needs attention, Completed, All, Attendance, Invoices, Receipts" and 5 summary cards	Matches `StudentsPage` tabs and `mini-stats` (cards: students in the tab, fees charged, fees received, balance due, plus a fifth for the active filter) [C]
Edit invoice: amount edit shifts Fee Total; status derived from receipts	Matches `InvoiceEditModal` and `invoiceSettlement`; the Profile invoice list is the exception (F4)
Edit receipt: per-course split, amount, paid-on, mode, reference, re-send checkbox	Matches `StudentReceiptModal` in edit mode (`receiptPreview` table)
Profile: counts, fee total, paid, balance, agreed fee, auto status, discount, coupon code	Matches `StudentDetailModal` Fee Tracking; the coupon field is a validator that applies the discount, not a stored free-text code field
Courses tab "actions dropdown (Renew cycle, Attendance sheet, add other charges, delete student, close/withdraw, WhatsApp confirmation)"	In code these are **separate controls**: Renew cycle and Attendance sheet are tab-level buttons; + Add other charges and the WhatsApp strip are tab content; Delete / Close are in the footer; the per-course **Actions** menu holds Complete, Certificate, Mark issued, review, batch, level, discontinue, remove

---

Appendix A — File index for the next session

Area	Files
Students screens	`src/pages/StudentsPage.jsx` (list L5411; detail L195; AddStudentModal L3527; receipts L4558/L4877; invoices L4979/L5306)
Student money engines	`src/utils/feeStatement.js`, `studentLedger.js`, `studentReceipts.js`, `components/studentDocs.js`, `components/StudentLedgerView.jsx`
Lifecycle / cycle / attention	`src/utils/studentLifecycle.js`
Attendance	`src/components/AttendanceRegister.jsx`, `AttendanceSheet.jsx`; `BatchesPage.BulkAttendanceModal`; `InstructorsPage.AttendanceModal`
Certificates	`src/components/StudentCertModal.jsx`, `public/certificate/*`, `api/send-cert-whatsapp.js`, `api/send-email.js › buildStudentCert`
WhatsApp / email	`src/services/whatsapp.js`, `services/email.js`, `api/send-whatsapp.js`, `send-payment-whatsapp.js`, `send-review-whatsapp.js`, `whatsapp-webhook.js`, `send-email.js`, `docs/whatsapp-templates.md`, `components/WhatsAppSendConfirm.jsx`, `utils/captureReceipt.js`
Franchisee onboarding	`src/pages/AccessRequestsPage.jsx`, `OnboardingPage.jsx`, `FranchiseesPage.jsx`, `api/create-user.js`, `api/_auth.js`
Roles / permissions	`src/constants/roles.js`, `constants/permissions.js`, `context/AuthContext.jsx`, migrations `20260519*`, `20260921113855_staff_granular_permissions.sql`
Schema	`supabase/migrations/` (160 files); key: `20260615110707_student_invoices`, `20260720123652_receipt_series_per_centre`, `20260728095502_student_fee_events_audit_log`, `20261010102551_student_receipt_date_and_renumber`, `20261010110252_student_no_class_days`, `20261010121603_instructor_attendance`, `20261012*` (payroll extras, conveyance, office salary)
Design docs	`CHECKPOINT.md` (2026-05-16, partly dated), `docs/transaction-model-migration-plan.md`, `docs/whatsapp-templates.md`

Appendix B — How to re-verify

- Row counts and the F2 figure: SQL in the audit session — `students` vs `enrollments` aggregate; re-run any time.
- Mirror coverage (F6): `grep -n "mirrorStudentToTransaction(" src/pages/StudentsPage.jsx` and compare with the functions listed.
- Defaults in F13: `grep -n "attended" src/pages/BatchesPage.jsx src/pages/InstructorsPage.jsx`.

- Step counts (§6.4) are derived from the component trees named in §6.2, not from timed sessions.